

Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

25/07/2022

Last (family) Name _____

First (given) Name _____

Matricola _____

Codice Persona	
-----------------------	--

Professor: ☐ Zanero ☐ Carminati ☐ Barenghi

Have you done any challenges/homework, even partially? ☐ Yes ☐ No

Do you want to withdraw? ☐ Yes ☐ No

Priority Grading ☐ Yes, Motivation:

[] No

Instructions

- **Duration:** 2.5 hours.
- The exam is composed of 19 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book". The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**

- On the computer, you can open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
- You will have to share your screen, leave your microphone opened, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct exam delivery.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or for a subset, at the instructor's decision.
- Regarding the submission:
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
 - We will evaluate only the uploaded **readable** documents.
 - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

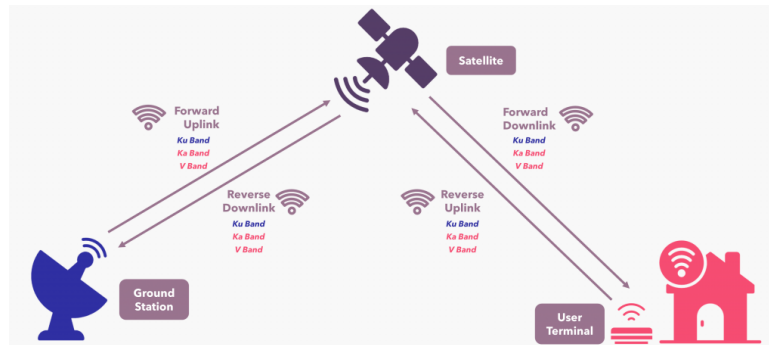
READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

STARKlink is a STARK initiative to create a global broadband network to bring global consumer-grade Internet access by using a constellation of Low Earth Orbit (LEO) satellites launched into space via rocket. They provide a two-way broadband service by simultaneously sending data to - and receiving data from - the ground station and the user terminal.

- The **User Terminal** is a small satellite dish that sits outside the user's home, ensuring a connection with the appropriate satellites moving across the sky.
- **Satellites** beam the instructions to a ground station, which processes the request.
- The **ground station** is connected via fiber to the Internet. Once requests are received by the ground station, the process is reversed as the ground station then sends the necessary data to the satellite (i.e., response) through the forward uplink and then back to the user terminal through the forward downlink.

Answer the questions below by considering the whole infrastructure that enables its functioning.



[1 point] 1. What are the main assets at risk in such a scenario? Suggest at least two assets.

Asset	Motivation
1) User data	...
2) Company Brand/ Internet Connectivity	...

[2 points] 2. Suggest, in rough order of prevalence (i.e., frequency), the most likely potential threats and threat agents against such infrastructure and their operating companies.

Threats	Threat Agents
1) Denial of Service Motivation: ...	1) Competitor Motivation: ...

2) Sniffing/Spoofing Motivation: Steal data and info	2) Motivated Black Hat Hacker Motivation: Steal data and info for economic advantage
---	---

[1 point] 2. Suggest at least two potential attack surfaces for the infrastructure under analysis.

Attack Surface	Motivation
1) Physical Endpoints (terminal or ground station)	...
2) Communication channels	...

Exercise 2 - Introduction to Cryptography (6 points)

Consider the case of a money transfer system. Each money transfer is represented as a text string written on a centralized ledger, held by a bank. An user of the system, to request a money transfer, writes a string with the recipient of the transfer, and the amount to be transferred, and authenticates it with a MAC, appending the tag. Users can only append new bank transfers, they are not allowed to read or overwrite previous transfers. At the end of each day, the bank checks the MACs and transfers the money to the recipients. As a receipt, the bank also sends the money transfer request (recipient, amount and tag) to the actual recipient, who is able to verify the MAC of the message.

[3 points] Is the system at hand preventing unauthorized money transfers? If yes, argue on why it can be relied upon; if no, describe an attack and appropriate countermeasures.

No. Since any user is able to verify MACs, this implies that any user is also able to compute MAC tags for the same transactions. This in turn allows a bank transfer recipient to add any number of extra transfers still with him as a recipient. The appropriate primitive to be used in this case is a cryptographic signature, as the actors verifying it should not be able to produce a valid one.

Alice and Bob want to communicate confidentially employing a symmetric encryption scheme. To this end, they decide to employ a Diffie-Hellman key exchange to agree on a symmetric session key. Knowing that the DH key exchange is vulnerable to active attacks tampering with the messages being sent, they decide to append to each message the digest of a secure cryptographic hash (think SHA-3-512, if you wish). Both Alice and Bob, when receiving a message, check that it has not been tampered by computing its digest and comparing it with the digest they received.

[3 points] Is the system at hand preventing active attacks? If yes, argue on why it can be relied upon; if no, describe an attack and appropriate countermeasures.

No. It is straightforward for an attacker to act as a man in the middle, since the only thing checked by Alice and Bob is a digest obtained from public material only, and without any signatures. A MITM can be performed by Oscar simply by impersonating Bob with Alice, and Alice with Bob, and taking care of adding the digest of the hash to his own Diffie-Hellman key agreement messages. A signature with an authenticated public key should be used by both Alice and Bob on their public DH messages.

Exercise 3 - Binary Vulnerabilities (6 points)

```
00 #include <stdio.h>
01 #include <stdlib.h>
02 #include <string.h>
03 #include <stdint.h>
04
05 typedef struct {
06     char buf[40];
07     char sel;
08 } data_t;
09
10 void reverse_string(data_t * data) {
11     char prefix[8];
12
13     scanf("%16s", prefix);
14     sprintf(data->buf, "%s\n", strrev(data->buf));
15     if (strncmp(data->buf, "DAVINCI:", 8) == 0) {
16         printf("%s", prefix);
17         printf(data->buf);
18         return;
19     }
20     else {
21         abort();
22     }
23 }
24
25 void main(int argc, char** argv) {
26     data_t data;
27
28     clearenv(); // Clear environment variables
29
30     scanf("%40s", data.buf);
31     scanf("%c", &data.sel);
32
33     if (data.sel == 'r') {
34         reverse_string(&data);
35     } else {
36         printf("%s", data.buf);
37     }
38 }
```

- The C standard library is loaded at a known address during every execution of the program, and that the address of the function `system()` is `0xf4d0e2d3`.
- Environment variables are located in the highest addresses.
- *The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdecl** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against exploiting memory corruption errors are present (unless specified otherwise).*
- **strrev () Reverses the given string**

[1 point] 1. The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

Vulnerability	Line	Motivate and Modify the line to solve the detected vulnerability
Buffer Overflow	13	[see slides]
Format String	17	[see slides]

[2 points] 2. **Focus only on the stack-based buffer overflow(s) you found.** Write an exploit for this vulnerability that **must execute the following shellcode**, composed of 8 bytes, which opens a shell: **0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90**.

2.a) Describe **all the steps and assumptions** required to successfully exploit the vulnerability. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution, if multiple executions are necessary.

- *Insert the (reversed) exploit (NOP+shellcode) in buf that must end with :ICNIVAD (when reversed DAVINCI:)*
- *select 'r'*
- *overflow prefix to overwrite sEIP with addr of ~buf+12*

ENV VARIABLE are cleaned at each execution -> placing shellcode in the env is not feasible
ALT SOL: directly exploit prefix

2.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

Before the vulnerable line				After the vulnerable line		ALTERNATIVE	
Low addresses							
^			Function frames				
	prefix[0-3]	<- ESP	reverse_string()	not_relevant		0x20 0x30 0x40 0x50	<-X
	prefix[4-7]		reverse_string()	not_relevant		0x60 0x70 0x80 0x90	
	sEBP	<- EBP	main()	not_relevant		not_relevant	
	sEIP		main()	~X		~X	
	&data		main()	not_relevant		not_relevant	
	buf[0-3]		main()	not_relevant		not_relevant	
	buf[4-7]		main()	not_relevant		not_relevant	
	buf[8-11]		main()	not_relevant	<-X	not_relevant	
	buf[12-15]		main()	not_relevant		not_relevant	
	buf[16-19]		main()	not_relevant		not_relevant	
	buf[20-23]		main()	not_relevant		not_relevant	
	shellcode		main()	shellcode		not_relevant	
	reverse		main()	reverse		not_relevant	
	:ICN		main()	:ICN		:ICN	
	IVAD		main()	IVAD		IVAD	
	sel (r)		main()	sel (r)		sel (r)	
	sEBP		main()	not_relevant		not_relevant	
	sEIP			not_relevant		not_relevant	
	argc			not_relevant		not_relevant	
	argv			not_relevant		not_relevant	
High addresses							

[2 points] 3. **Focus only on the format string(s) you found.**

3.a) Write an exploit for this vulnerability that executes the previous shellcode, assuming that you have already prepared the memory (the shellcode has been positioned in a place under the control of the attacker) with the correct arguments. Assume that:

- The address of the return address (*saved EIP*) of the function exploited is: **0x8da0fee4** (i.e., where to write)
- The address of the first instruction of the **shellcode** is at **0xd3f4e2d0** (i.e., what to write)
- The displacement on the stack of the vulnerable function's argument is: **7**

Knowing that $\text{dec}(0xd3f4) = 54260$ and $\text{dec}(0xe2d0) = 58064$, write the exploit clearly, describe **all the steps, assumptions and the components of the format string** required to successfully exploit the vulnerability. Include also any assumption on how you must call and run the program; e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution.

- *Insert the (reversed) FS exploit in buf and must end with :ICNIVAD (when reversed DAVINCI:)*
- *select 'r'*
- *Exploit executed to overwrite sEIP with addr of shellcode*
- *EXPLOIT AS SEEN AT LECTURE -> + 8 (DAVINCI:) char already printed!*

$\text{dec}(0xd3f4) = 54260 < \text{dec}(0xe2d0) = 58064$

Swap required

<target+2><target>%<lower_part-16>c%pos+2%n<higher_part-low_part>c%pos+2+1%n

+ [Substitute the values and make computation]

3.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

prefix[0-3]	not_relevant
prefix[4-7]	not_relevant
sEBP	not_relevant
sEIP	0xd3f4e2d0
&data	not_relevant
buf[0-3]	not_relevant
buf[4-7]	not_relevant
buf[8-11]	not_relevant
buf[12-15]	not_relevant
buf[16-19]	not_relevant
buf[20-23]	not_relevant
reverse	reverse
FS exploit	FS exploit
:ICN	:ICN
IVAD	IVAD
sel (r)	sel (r)
sEBP	not_relevant
sEIP	not_relevant
argc	not_relevant
argv	not_relevant

[1 point] 4. Consider ONLY the correctly implemented “Random Terminator Canary” mitigation technique (i.e, randomly generated at each execution with ‘\0’ in its value). Is it effective to prevent your

exploit at point 2. And 3. ? If the mitigation technique is effective, explain why and describe how you would bypass the mitigation in order to **execute the same shellcode of the previous question.**

It is effective to prevent the exploit at point 2 but not the exploit at point 3.

+ motivation [see slides].

The only way to bypass the mitigation is to use the FS. With the BO it is not possible since it is limited to 16 bytes.

Exercise 4 - Web Vulnerabilities (6 points)

Retro Cinema Club is an online forum for old movie lovers. Users can review movies with stars (0 to 5) and share their thoughts about a movie with an optional comment. Moreover, the recommender system of the website can suggest to the users unwatched movies that are very close to their taste.

Details:

- For each movie, there is a *dedicated web page* with all the information about the movie and a list of reviews. By clicking the movie icon in the **/home** page of the website, it is possible to reach its dedicated page.
- If a movie is not present in the list of reviewable movies, the user can request an admin to add it to the list. This is done through a *request movie* page which collects in the database all the requests of the users.
- There is no user support, i.e., it is not possible for the user to contact the admin directly if not via the request movie option. However, if the Title or the Movie Director of a request contains a link, the admin will drop the request (i.e., they will not add the movie to the list).
- The admin accepts the request if the Title, the Movie Director, and the year are consistent. This check is performed manually by the admin.

For instance:

- Title: "The Good, The Bad And The Ugly", Movie Director: "Sergio Leone", year: **1966**
-> ACCEPT
- Title: "The Good, The Bad And The Ugly", Movie Director: "Sergio Leone", year: **2006**
-> DENY

The relevant code of the application is the following:

```
01 app.route("/login") # /login
02 def login(request, session):
03     if not is_session_valid(session):
04         return redirect(url_for("/home"))
05     if request.method != "POST":
06         return abort(403)
07     username = request.form["username"]
08     password = request.form["password"]
09     hash_password = SHA256_hash(password) # Meh!
10     query_st = "SELECT user_id, is_admin FROM USERS WHERE username = %s AND
    password = %s;"
```

```
11     result = db_execute_query_with_statement(query_st, username, hash_password)
12     if len(result) == 1:
13         is_admin = result[0]["is_admin"]
14         if is_admin == True:
15             session["level"] = "admin"
16         else:
17             session["level"] = "user"
18             session["user_id"] = result[0]["user_id"]
19         return redirect(url_for("/home"))
20     else:
21         return abort(403)
22
23 app.route("/add_movie") # /add_movie
24 def add_movie(request, session):
25     if not is_session_valid(session):
26         return redirect(url_for("/login"))
27     if request.method != "POST":
28         return abort(403)
29     if session["level"] != "admin":
30         return abort(401)
31     title = XSS_sanitizer(request.form["title"])
32     md = XSS_sanitizer(request.form["movie_director"])
33     year = XSS_sanitizer(request.form["year"])
34     query_st = "SELECT movie_id FROM MOVIES WHERE title = %s AND movie_director = %s AND year = %s;"
35     result = db_execute_query_with_statement(query_st, title, md, year)
36     if len(result) == 1:
37         movie_id = result[0]["movie_id"]
38     else:
39         insert_st = "INSERT INTO MOVIES VALUES (%s, %s, %s);"
40         db_execute_query_with_statement(insert_st, title, md, year)
41         result = db_execute_query_with_statement(query_st, title, md, year)
42         movie_id = result[0]["movie_id"]
43     return redirect(url_for("/movie", movie_id=movie_id)) # Visiting the page of the movie
44
45 app.route("/review") # /review
46 def review(request, session):
47     if not is_session_valid(session):
48         redirect(url_for("/login"))
49     if request.method != "POST":
50         abort(403)
51     movie_id = request.form["movie_id"]
52     user_id = session["user_id"]
53     comment = request.form["comment"]
54     stars = request.form["stars"]
55     date = time.now()
56     insert_st = "INSERT INTO REVIEWS VALUES (%s, %s, %s, %d, %s);"
57     db_execute_query_with_statement(insert_st, user_id, movie_id, comment, stars, date) # If the query fails, don't care
58     return redirect(url_for("/movie", movie_id=movie_id)) # Visiting the page of the movie
59
60 app.route("/movie") # /movie
61 def movie(request, session):
62     if not is_session_valid(session):
63         return redirect(url_for("/login"))
64     if request.method != "GET":
65         return abort(403)
66     movie_id = request.args["movie_id"]
67     query_1 = "SELECT * FROM MOVIES WHERE movie_id = %s;"
68     query_2 = "SELECT * FROM REVIEWS WHERE movie_id = %s;"
69     movie_det = db_execute_query_with_statement(query_1, movie_id)
```

```
70     reviews = db_execute_query_with_statement(query_2, movie_id)
71     return render_template("movie.html", movie_det=movie_det, reviews=reviews)
72
73 # movie.html
74 <html>
75     ...
76     <h1>{{ movie_det[0]["title"] }}</h1>
77     <h2>{{ movie_det[0]["movie_director"] }}</h2>
78     <h2>{{ movie_det[0]["year"] }}</h2>
79     ...
80     {% for review in reviews %} # Iterating over reviews
81         <p>{{ review["comment"] }}</p>
82     {% endfor %}
83     ...
84 </html>
```

The relational database schema used by the website is the following:

Table: **USERS**

user_id (Primary Key) (String) (Auto increment)	username (String)	email (String)	password_hash (Bytes)	is_admin (Boolean)
---	----------------------	-------------------	--------------------------	-----------------------

Table: **MOVIES**

movie_id (Primary Key) (String) (Auto increment)	title (String)	movie_director (String)	year (Date)
--	-------------------	----------------------------	----------------

Table: **REVIEWS**

user_id (Primary Key) (Referencing USERS.UserID)	movie_id (Primary Key) (Referencing MOVIES.MovieID)	comment (String)	stars (Int)	date (Datetime)
---	---	---------------------	----------------	--------------------

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **XSS_sanitizer** correctly escapes all the html entities, i.e., all the html special characters are converted to their equivalent data (> becomes >).
- The function **db_execute_query_with_statement** correctly executes queries with prepared statements.
- The function **db_execute_query_with_statement** returns a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, `result[4]["comment"]` is accessing the field "comment" of the 5th row.
- The function **redirect** redirects the user to the web page of the provided url.
- The function **url_for** generates an url of the provided page of the same domain. Optional query parameters of the url can be specified as parameters.
- The function **render_template** generates an html page with optional parameters that are placed in the html code. **It does not perform any sanitization of the input.**

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

<i>Vulnerability class</i>	<i>Is there a vulnerability of this class in the code above? How could an adversary exploit it?</i>	<i>If the vulnerability is present, explain the simplest procedure to remove it.</i>
Sql Injection (SQL-I)	<i>No, all queries are executed through a prepared-statement invoking function</i>	<i>none</i>
cross-site scripting (XSS) Specify if reflected, stored, DOM...	<i>Yes, the comment of a review is not sanitized. When the user reviews a movie, he/she can insert arbitrary javascript in the comment of the review.</i>	<i>The simplest procedure would be to apply escaping/filtering to the comment of the review (e.g., the XSS_sanitizer function which is available).</i>
Cross site request forgery (CSRF)	<i>Yes, there is no implementation of a CSRF token.</i>	<i>See slides (CSRF-tokens, etc.).</i>

[1.5 points] 2. One of your friends sent you a link to the page of the movie "Forrest Gump", but once you click the link, an alert pops up saying: "Run Forrest, run!!!". Can you reproduce the exploit? Explain all the steps you'd take to reproduce the exploit, including the final code you'd write.

To reproduce the exploit you just need to insert a review to the page of the movie "Forrest Gump" with the following script as comment: `<script>alert("Run Forrest, run");</script>`

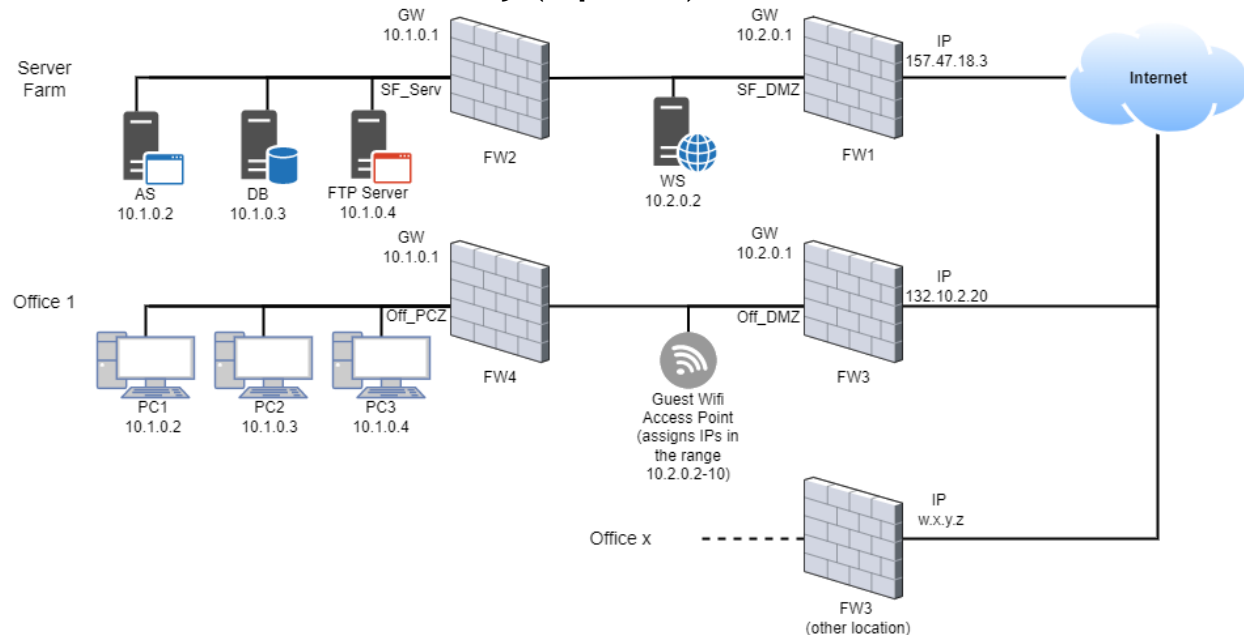
[1.5 points] 3. Is it possible to log in as an admin by exploiting one of the identified vulnerabilities? If yes, please explain all the steps of the exploit. If not, please explain why it is not possible to do so. Remember, the only option to reach an admin is through the request movie page. Moreover, he won't click any link you can provide through that page.

Yes, we can exploit the XSS vulnerability to make the admin post its session cookie as a comment of a review.

The exploit is the following:

- 1) Choose a movie page for the attack (movie X)*
- 2) Insert the XSS that posts the session cookie as a comment of another movie (movie Y)*
- 3) Request the movie X once again, so the admin will visit the page in which the exploit is present.*
- 4) Collect the session cookie in the page of the movie Y*

Exercise 5 - Network Security (6 points)



A multinational animal video sharing company, PetFlix, has offices in various parts of the world and a small server farm. The offices are used by the employees (in the image only one office is fully displayed, although multiple ones exist), while the server farm stores the **web** and **application** servers, the **database**, and an **FTP** server used by the employees to access and upload the videos then available to the clients by the web server.

The **PCx** and **Guest WiFi** Access Point in the offices require web access to the internet with http & https, while the **web** server requires to be accessible through http & https, and needs access to the **application** server (on custom port 30) that in turn communicates with the database. The **PCx** also require access to the **FTP** server.

All the local networks are **NATted**, and present to the Internet the IP shown outside **FW1** and **FW3** (for the FW rules, you can consider Office 1 only).

The **FTP** server allows only for **passive** mode, which means that the FTP client chooses the data port (for the sake of simplicity, assume that the data port can be chosen only in the port range 500-999).

Both Source and Destination NAT (for both local networks) use the following (relevant) associations:

For the server farm:	For the offices:
10.1.0.1:1-999 = 157.47.18.3:1001-1999	10.1.0.1:1-999 = 132.10.2.20:1001-1999

10.1.0.2:1-999 = 157.47.18.3:2001-2999 ... up to 9 that goes to 9001-9999 10.2.0.1:1-999 = 157.47.18.3:21001-21999 10.2.0.2:1-999 = 157.47.18.3:22001-22999 ... up to 9 that goes to 9001-9999	10.1.0.2:1-999 = 132.10.2.20:2001-2999 ... up to 9 that goes to 9001-9999 10.2.0.1:1-999 = 132.10.2.20:21001-21999 10.2.0.2:1-999 = 132.10.2.20:22001-22999 ... up to 9 that goes to 9001-9999
--	--

FW	Src IP	Src PORT	Direction of the 1st packet	Dst IP	Dst PORT	Policy	Description
1	Any	Any	Int->DMZ	157.47.18.3	22080/22443	Allow	Access for everyone to the Web Server
1	132.10.2.20	1000-4999	Int->DMZ	157.47.18.3	4021	Allow	Access for PCx to FTP Command port
2	10.2.0.2	Any	DMZ->SF_Serv	10.1.0.2	30	Allow	Access for the WS to the AS
2	132.10.2.20	1000-4999	DMZ->SF_Serv	10.1.0.4	21	Allow	Access for PCx to FTP Command port
2	10.1.0.4	Any	SF_serv->DMZ	10.2.0.2	80/443	Allow	Access for the FTP server to the WS
3	10.2.0.2-240	Any	DMZ->Int	Any	Any	Allow	Web surfing access+ WS for Guests
3	10.1.0.2-4	Any	DMZ->Int	Any	80/443	Allow	Access for PCx to Internet and WS
3	10.1.0.2-4	Any	DMZ->Int	157.47.18.3	4021	Allow	Access for PCx to FTP Command port
4	10.1.0.2-4	Any	Off_PCZ->DMZ	Any	80/443	Allow	Access for PCx to Internet and WS
4	10.1.0.2-4	Any	Off_PCZ->DMZ	157.47.18.3	4021	Allow	Access for PCx to FTP Command port

[2 points] 1. Considering that the company firewalls are **stateful**, are the **firewall rules** presented above complete and correct to obtain the effects presented in the description? Write below the ones that should be added, removed, or modified.

Default Deny Missing
FTP Data Port Rules are missing
- Rule regarding communication between FTP and WS useless
6) Modify rule for Guest to access internet only with 80/443

[2 points] 2. Eventually, the FTP server becomes unreachable. Can you guess what happened, explaining how the attack works and where the attacker is, given the relevant network logs shown below? (to help the comprehension, the timestamps ignore delay, so that a packet that starts in location A and gets to location B will be shown at the same timestamp in both locations).

SF_Serv and SF_DMZ:

Timestamp	Src IP:port	Dst IP:port	Description
+00,000s	132.10.2.20:3021	10.1.0.4:21	TCP SYN
+00,001s	10.1.0.4:21	132.10.2.20:3021	TCP SYN ACK
+00,002s	132.10.2.20:3021	10.1.0.4:21	TCP SYN
+00,003s	10.1.0.4:21	132.10.2.20:3021	TCP SYN ACK
+00,004s	132.10.2.20:3021	10.1.0.4:21	TCP SYN
+00,005s	10.1.0.4:21	132.10.2.20:3021	TCP SYN ACK
+00,006s	132.10.2.20:3021	10.1.0.4:21	TCP SYN
+00,007s	132.10.2.20:3021	10.1.0.4:21	TCP SYN
+00,008s	132.10.2.20:3021	10.1.0.4:21	TCP SYN
+00,009s	132.10.2.20:3021	10.1.0.4:21	TCP SYN
	Continues similarly		

Off_DMZ:

Timestamp	Src IP:port	Dst IP:port	Description
+00,000s	10.1.0.3:21	132.10.2.20:4021	TCP SYN
+00,001s	132.10.2.20:4021	10.1.0.3:21	TCP SYN ACK
+00,002s	10.1.0.3:21	132.10.2.20:4021	TCP SYN
+00,003s	132.10.2.20:4021	10.1.0.3:21	TCP SYN ACK
+00,004s	10.1.0.3:21	132.10.2.20:4021	TCP SYN
+00,005s	132.10.2.20:4021	10.1.0.3:21	TCP SYN ACK
+00,006s	10.1.0.3:21	132.10.2.20:4021	TCP SYN
+00,007s	10.1.0.3:21	132.10.2.20:4021	TCP SYN
+00,008s	10.1.0.3:21	132.10.2.20:4021	TCP SYN
+00,009s	10.1.0.3:21	132.10.2.20:4021	TCP SYN
	Continues similarly		

Off_PCZ:

Timestamp	Src IP:port	Dst IP:port	Description
	Empty		

SYN flood [see slides]

The attacker is in the guest wifi, spoofing the IP of a PC from Off_PCZ

[2 points] 3. Assume that the communication between the PCx and the FTP server is not encrypted. Can an attacker (a) located in the Guest WiFi network or (b) that has control of the Gateway of the subnetwork “Off_PCZ” implement a MITM attack? Would the answer change if all the MAC addresses of the network devices (i.e, gateways, switches, routers, FWs) are hardcoded and the Cisco PORT SECURITY is enabled? Consider all network equipment and devices connected through switches. Justify your responses.

MAC addresses **not** hardcoded

(a) **Yes, by ARP spoofing the gateway**

(b) **Yes, it is the gateway :P**

MAC addresses hardcoded

(a) **No, sniffing -> port security; No ARP spoof -> hardcoded**

(b) **Yes, it is the gateway :P**

Exercise 6 - Malware (6 points)

Tomb-Bino is a novel malware family that spreads through email attachments, turning infected computers into peers of a command and control infrastructure. An attacker can then “command” an infected computer remotely, for instance, to perform DDoS, steal data, etc.

Its body comprises three functions, which are loaded into memory when the victim clicks on the email attachment: **spread**, **mutate**, and **sit_and_wait**.

spread performs the following operations (in order):

1. Steals the list of the user’s contacts.
2. Calls **mutate**.
3. Creates a new file, “new_attachment.jpeg.exe”, whose instructions are dumped from the process memory.
4. Sends “new_attachment.jpeg.exe” to every user in the contact list as an email attachment.

mutate takes as input (**start_address**, **number_of_bytes**) and substitutes each instruction found in memory at [**start_address**, (**start_address** + **number_of_bytes**)] with a semantically equivalent one. For instance, instruction **sub eax, 0x1** is substituted with **add eax, -0x1**. Substitutions are performed directly to the memory.

Lastly, **sit_and_wait** simply waits for commands from the server and executes them.

The following is a simplified view of the process memory when a Tomb-Bino instance is executed:

```
0x00100000 <spread>
(1000 bytes long)
0x001003e8 <sit_and_wait>
(2000 bytes long)
0x00100bb8 <mutate>
(4000 bytes long)
```

6.1 [2 points] What type of class does this malware most likely belong to (encrypted, polymorphic, metamorphic, evasive)? Motivate your answer.

Metamorphic

mutate produces semantically equivalent but “different” code

6.2 [2 points] Suppose that the headers (i.e., library imports, file details, etc) of each “new_attachment.jpeg.exe” cannot be used to generate signatures. Moreover, suppose that **spread** invokes **mutate** with parameters (**0x00100000, 3000**). Could this family of malware be detected through signatures? If yes, explain why. If not, propose a different detection strategy.

Yes, it can be detected through signatures.

When the malware mutates (i.e., when the function mutate is invoked), it only changes the first two functions, leaving the mutation engine (i.e., mutate) unchanged. Therefore, an antivirus could use mutate as a target for its signatures.

4.3 [2 points] Given the nature of Tomb-Bino (i.e., a command and control trojan that spreads via email), how would you employ dynamic analysis to detect it? In particular, what kind of resources or operations would you monitor? (Note: a description of command and control malware is provided at the beginning of this section).

1) Email access (retrieve contacts, send emails)

2) General networking operations (listen on a port, sending/receiving data to specific IPs)